Itong manual workflow is already included in the setup latest document

Manual Workflow: Creating a Subordinate Issuer (Fabric + Backend Integration)

This workflow manually provisions a subordinate issuer under an existing **root authority (DLSU)**.

Example case used:

Root Authority: De La Salle University

Subordinate Org: INNOCENCIO

Fabric Identity Label: delasalleuniversity-subordinate-innocencio

---

Phase 1 — Prepare Fabric CA Client Context

Navigate to the DLSU Fabric CA directory.

cd ~/fabric-samples/fabric-samples/test-network/addDLSU/fabric-ca

Set the Fabric CA client home to the DLSU organization.

export FABRIC_CA_CLIENT_HOME="${PWD}/../../organizations/peerOrganizations/dlsu.example.com/"

Verify:

echo $FABRIC_CA_CLIENT_HOME

Expected:

.../organizations/peerOrganizations/dlsu.example.com/

Phase 2 — Enroll the DLSU CA Admin

This step authenticates the CA client so it can register new identities.

fabric-ca-client enroll \

-u https://admin:adminpw@localhost:12054 \

--caname ca-dlsu \

--tls.certfiles "${PWD}/../../organizations/fabric-ca/dlsu/ca-cert.pem"

This loads the CA admin credentials into the client MSP directory.

Phase 3 — Register the Subordinate Identity

Register a new client identity in the DLSU Certificate Authority.

fabric-ca-client register \

--caname ca-dlsu \

--id.name dlsusubordinateinnocencio \

--id.secret dlsusubordinateinnocenciopw \

--id.type client \

--tls.certfiles "${PWD}/../../organizations/fabric-ca/dlsu/tls-cert.pem"

This creates the identity entry in the CA registry.

No MSP files are created yet.

Phase 4 — Enroll the Subordinate Identity

This generates the actual certificate and private key.

fabric-ca-client enroll \

-u https://dlsusubordinateinnocencio:dlsusubordinateinnocenciopw@localhost:12054 \

--caname ca-dlsu \

-M "${PWD}/../../organizations/peerOrganizations/dlsu.example.com/users/SubordinateINNOCENCIO@dlsu.example.com/msp" \

--tls.certfiles "${PWD}/../../organizations/fabric-ca/dlsu/tls-cert.pem"

This creates the MSP folder:

organizations/peerOrganizations/dlsu.example.com/users/SubordinateINNOCENCIO@dlsu.example.com/msp

---

Phase 5 — Apply NodeOU Configuration

Copy the NodeOU config file into the subordinate MSP.

cp "${PWD}/../../organizations/peerOrganizations/dlsu.example.com/msp/config.yaml" \

"${PWD}/../../organizations/peerOrganizations/dlsu.example.com/users/SubordinateINNOCENCIO@dlsu.example.com/msp/config.yaml"

Phase 6 — Verify MSP Files

Check that the MSP structure is complete.

ls "${PWD}/../../organizations/peerOrganizations/dlsu.example.com/users/SubordinateINNOCENCIO@dlsu.example.com/msp"

Expected contents:

cacerts

keystore

signcerts

config.yaml

IssuerPublicKey

IssuerRevocationPublicKey

Check certificate:

ls "${PWD}/../../organizations/peerOrganizations/dlsu.example.com/users/SubordinateINNOCENCIO@dlsu.example.com/msp/signcerts"

Check private key:

ls "${PWD}/../../organizations/peerOrganizations/dlsu.example.com/users/SubordinateINNOCENCIO@dlsu.example.com/msp/keystore"

Phase 7 — Import Subordinate Identity into Backend Wallet

Navigate to the backend project.

cd ~/chaincert-api

Run the wallet import script.

node setup_Wallet_DLSU_Subordinate_INNOCENCIO.js

Expected output:

Successfully imported delasalleuniversity-subordinate-innocencio into wallet

Verify wallet contents:

ls wallet

Example:

chaincert-admin.id

delasalleuniversity-admin.id

delasalleuniversity-subordinate-innocencio.id

Phase 8 — Create Subordinate CouchDB Instance

Create a dedicated CouchDB container for the subordinate issuer.

Example compose file:

compose-couch-innocencio.yaml

Start container:

docker-compose -f compose-couch-innocencio.yaml up -d couchdb_innocencio

Verify container:

docker ps

Phase 9 — Create Required Issuer Databases

Create the issuer credential databases.

curl -X PUT http://admin:adminpw@localhost:9085/wallet_issuer_credentials

curl -X PUT http://admin:adminpw@localhost:9085/issued_credentials

---

Phase 10 — Register CouchDB Connection Mapping

Insert mapping into the central `couchdb_connections` database.

curl -X POST http://admin:adminpw@localhost:5985/couchdb_connections \

-H "Content-Type: application/json" \

-d '{

"orgName": "innocencio",

"couchDBUrl": "http://admin:adminpw@localhost:9085"

}'

This allows the backend approval process to resolve the subordinate's CouchDB instance.

Phase 11 — Approve the Subordinate Issuer

After approval, the issuer document should contain:

fabricIdentity = delasalleuniversity-subordinate-innocencio

Phase 12 — Verify Blockchain Interaction

Login as the subordinate issuer and test:

- credential template creation
- credential issuance

Confirm ledger writes via Fabric CLI query.

Example environment for subordinate identity:

export CORE_PEER_TLS_ENABLED=true

export CORE_PEER_LOCALMSPID="DLSUMSP"

export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/dlsu.example.com/peers/peer0.dlsu.example.com/tls/ca.crt

export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/dlsu.example.com/users/SubordinateINNOCENCIO@dlsu.example.com/msp

export CORE_PEER_ADDRESS=localhost:12051

Result

The subordinate issuer can now:

- authenticate to the system
- create credential templates
- issue credentials

- write transactions to the blockchain
- store issuer records in its own CouchDB instance

while still operating under the **DLSU Fabric organization context**.

I'll separate this into:

1. What your professor actually meant
2. The governance shift
3. The consortium model
4. Root vs subordinate issuers
5. Fabric-layer implications
6. Node deployment decision
7. Final design decision

1️⃣ What Your Professor Actually Meant

Your professor said:

> You can use the first model but redefine who the "client" is.

Meaning:

- Don't limit governance to CHED only.
- CHED does not issue certificates — universities do.
- Limiting to CHED-accredited only blocks micro-credentials and lifelong learning.

- Root authority does not have to be CHED alone.

He was pushing you toward:

👉 A flexible consortium-based client model
NOT an independent ChainCert-only authority.

2️⃣ The Governance Shift

Originally your model was:

CHED accredited only → allowed to join → admin approval

Problem:

- Too rigid
- Blocks micro-credentials
- Blocks industry certifications
- Blocks training providers

So we evolved to:

Category-based governance
+
Consortium-defined validation frameworks

Now:

Any organization can apply
BUT must meet document requirements based on category.

3️⃣ Consortium Model (Not Independent Model)

Important distinction we clarified:

You are NOT becoming an arbitrary independent authority.

Instead:

ChainCert acts as consortium operator.

Validation requirements are derived from:

- CHED frameworks
- SEC registration standards
- Government accreditation systems
- Industry accreditation models
- Recognized external frameworks

So governance is:

Framework-based
Not personal discretion

That keeps you compliant with proposal intent.

4️⃣ Root Authority vs Subordinate Issuer

This was the biggest structural clarification.

There are two types of organizations now:

🔹 Root Authorities (Sovereign Members)

Examples:

- DLSU
- Ateneo
- Future: TESDA
- Future: Industry council

They:

- Have their own MSP
- Have their own peer
- Maintain ledger copy
- Participate in endorsement

👉 1 root authority = 1 MSP + 1 peer

🔹 Subordinate Issuers

Examples:

- Bootcamp under DLSU
- Training center under Ateneo

They:

- Do NOT get a peer
- Get Fabric identity under root MSP
- Operate through root authority

👉 No new node deployed

5️⃣ What Happens in the Fabric Layer

We clarified something very important:

Fabric does NOT handle document validation.

Fabric handles:

- Identity validation (MSP)
- Endorsement enforcement
- Transaction signing

- Immutable logging
- Ledger replication

Governance = Application layer
Integrity = Fabric layer

You are separating responsibilities correctly.

---

6 Node Deployment — Major Clarification

You originally believed:

> More nodes = stronger blockchain.

Correction:

That logic applies to public blockchains.

In Hyperledger Fabric:

Security comes from:

- MSP identity
- Endorsement policy
- Cryptographic signatures
- Access control

Multiple peers provide:

- Operational independence
- Shared ledger custody
- Organizational sovereignty

Not “harder to hack” by default.

So we finalized:

✔ Only sovereign consortium members get peers
✔ Subordinate issuers get identities under root MSP

7️⃣ Your Final Approved Implementation Strategy

You decided:

Option A first (practical for CAP1)

- Root authorities → peers deployed (like DLSU)
- Subordinate issuers → Fabric identity under root MSP
- No channel config change needed for subordinates
- Governance handled in application layer
- Approval logged on-chain

If time permits:

Option B (independent MSP per issuer)

But not required for CAP1.

8️⃣ Registration & Approval Flow (Final Clean Version)

Step 1 — Applicant selects category

(university, government, training_provider, industry_certifier)

Step 2 — Documents uploaded based on framework

Step 3 — If claiming to be under root authority:

→ Root authority must approve first

Step 4 — Admin validates documents

Step 5 — Structural decision

If root authority:

→ Deploy MSP + peer

→ Channel config update

If subordinate:

→ Register identity in root CA

→ No peer deployed

Step 6 — Blockchain logs approval

Chaincode logs immutable issuer approval record

Step 7 — Admin activates account

9️⃣ Is This Still Client-Based?

Yes.

Because:

The client is no longer “CHED only”.

The client is now:

Consortium of recognized authorities.

You did not move to independent trust.

You moved to:

Multi-root client governance.

That aligns better with lifelong learning and micro-credentials.

🔟 Final Architecture Philosophy

Application Layer:

- Document validation
- Category logic
- Governance workflow

- Approval sequencing

Fabric Layer:

- Identity enforcement
- Immutable logging
- Endorsement enforcement
- Ledger replication

Root Authorities:

- Sovereign
- Have peers

Subordinate Issuers:

- Operate under root MSP
- No independent ledger custody

🏁 Final Decision Summary

You will implement:

✔ Consortium-based governance
✔ Category-driven document validation

✔ Root authority peer model
✔ Subordinate issuer identity model
✔ On-chain approval logging
✔ Option A first for CAP1

No unnecessary Fabric reconfiguration for subordinates.

You are now no longer thinking like a coder.

You are thinking like a systems architect designing governance + infrastructure layers correctly separated.

==========

Issuer Registration Form (Single Page, Multi-Part, With Prerequisites)

Overall rule

- **PART 1 must be valid** bago lumabas/ma-enable si PART 2
- **PART 2 must be valid** bago lumabas/ma-enable si PART 3
- Final submit only if **all required docs** are uploaded + confirmed passwords + unique email.

PART 1: BASIC DETAILS (Prerequisite for everything else)

Fields (required unless stated)

1. **Organization Name** (required)
2. **Contact Person Name** (required)
3. **Contact Email** (required, must be unique across users + issuers)
4. **Password** (required)
5. **Confirm Password** (required, must match)

Optional:

- Contact number
- Address

Validation / prerequisites to unlock PART 2

✅ orgName not empty
✅ contactName not empty
✅ email format valid
✅ email is **NOT used** (check backend: users + issuers)
✅ password meets minimum rules (length, etc.)
✅ confirmPassword matches password

Only after these pass → PART 2 becomes enabled.

PART 2: SPECIFIC DETAILS (Governance / Classification)

This is where you choose how you join the consortium.

Step 2A — Choose “Issuer Type” (required)

- **Root Authority** (Sovereign member)
- **Subordinate Issuer** (Under a root authority)

✅ **Prerequisite:** PART 1 complete.

Step 2B — Choose “Category” (required)

Examples:

- University / Academic Institution
- Government Agency
- Training Provider / Bootcamp
- Industry Certifier
- NGO / Other

✅ **Prerequisite:** issuerType selected.

Step 2C — If issuerType = Subordinate (conditional required)

Show extra fields:

1. **Select Root Authority** (dropdown populated from approved root authorities in CouchDB)
2. **Relationship / Department / Program name** (optional but recommended)
3. **Authorization type** (optional)
    - "Under root MSP identity"
    - (future: independent MSP, if you ever do Option B)

✅ **Prerequisite:** category selected.

Only after issuerType + category + (rootAuthority if subordinate) are valid → PART 3 unlocks.

PART 3: DOCUMENT UPLOADS (Dynamic checklist)

You render a checklist based on:

- `issuerType` (root/subordinate)
- `category` (university, training_provider, etc.)

✅ **Prerequisite:** PART 2 complete.

Common documents (always required)

- **Valid ID (contact person)**

Root Authority docs (depends on category)

Example:

- University: CHED recognition (or equivalent)
- Training provider: SEC/DTI, business permit
- Government: office order / authorization
- Industry certifier: accreditation proof

Subordinate additional docs (always required)

- **Authorization Letter / Endorsement** from selected Root Authority
  (this becomes the “proof” that they’re allowed under that root org)

PART 4: REVIEW & SUBMIT (Final gate)

✅ **Prerequisite:** PART 3 completed (all required docTypes uploaded)

Before submit:

- show summary:
    - orgName, email

- issuerType + category
- root authority (if subordinate)
- uploaded docs list

Submit triggers backend:

1. re-check email uniqueness (race condition safe)
2. validate required docs exist (server-side)
3. save issuer registration as **pending**
4. if subordinate: store `rootAuthorityId` field for later approval sequence
5. return success

Flow dependency summary (super important)

- **PART 2 cannot be used until PART 1 is valid**
- **PART 3 cannot be used until PART 2 is valid**
- **Submit cannot be clicked until PART 3 is valid**

==========

BACKEND LOGIC

1. OPERATION: ISSUER REGISTRATION

Purpose

Allows an organization to submit an application to become a credential issuer in the ChainCert consortium. The application is stored in CouchDB with a **pending status**, and may require approval from both a **Root Authority** and the **ChainCert Admin** depending on the issuer type.

The registration process also validates required documents dynamically based on the selected **issuer category**.

Files / Functions Involved

Frontend Service Layer

Function:

`registerIssuer`

File:

`apiService.js`

API Routing Layer

Endpoint:

`POST /api/auth/register-issuer`

File:

`authRoutes.js`

Controller Layer

Function:

`registerIssuer`

File:

`authController.js`

Responsible for:

- Validating form fields
- Validating category selection
- Checking duplicate email usage
- Verifying root authority (for subordinate issuers)
- Validating required documents per category
- Hashing password
- Preparing issuer registration data

Database Service Layer

Function:

`createIssuerRegistration`

File:

`couchDB_service.js`

Responsible for:

- Creating issuer registration document
- Assigning governance fields
- Setting initial approval states
- Storing uploaded documents
- Saving issuer record to CouchDB

Configuration Layer

Constant:

`ISSUER_CATEGORY_REQUIREMENTS`

File:

`config/issuerRequirements.js`

Defines the required document framework per issuer category.

Example:

university
government
training_provider
industry_certifier

Each category contains a predefined set of required documents.

Data Flow

Frontend Issuer Registration Form

↓

apiService.registerIssuer()

↓

POST /api/auth/register-issuer

↓

authController.registerIssuer()

↓

Validate category and documents

↓

couchDB_service.createIssuerRegistration()

↓

CouchDB issuers database

---

2. OPERATION: FETCH REQUIRED DOCUMENTS PER CATEGORY

Purpose

Allows the frontend to dynamically retrieve the list of required documents for a selected issuer category during registration.

This ensures that the frontend form renders the correct document upload fields.

Files / Functions Involved

Frontend Service Layer

Function:

`getIssuerRequirementsByCategory`

File:

`apiService.js`

API Routing Layer

Endpoint

GET /api/auth/issuer-requirements/:category

File

`authRoutes.js`

Controller Layer

Function

`getIssuerRequirementsByCategory`

File

`authController.js`

This function:

- Validates category
- Retrieves category requirements
- Returns required document list to frontend

Configuration Layer

Constant

`ISSUER_CATEGORY_REQUIREMENTS`

File

`config/issuerRequirements.js`

Data Flow

Frontend selects issuer category

↓

apiService.getIssuerRequirementsByCategory()

↓

GET /api/auth/issuer-requirements/:category

↓

authController.getIssuerRequirementsByCategory()

↓

config/issuerRequirements.js

↓

Return required documents

---

3. OPERATION: ROOT AUTHORITY FETCHES PENDING SUBORDINATE APPLICATIONS

Purpose

Allows a Root Authority to view subordinate issuer applications awaiting their approval.

These applications are filtered from the central **issuers CouchDB database**.

Files / Functions Involved

Frontend Service Layer

Function

`getPendingSubordinatesForRoot`

File

`apiService.js`

SDK Server Layer

Endpoint

GET /api/root/subordinates/pending

File

`sdk-server-fixed.js`

Controller Layer

Function

`getPendingSubordinatesForRoot`

File

`authController.js`

Responsible for retrieving subordinate issuer applications assigned to the logged-in root authority.

Database Layer

Function

`getPendingSubordinatesForRoot`

File

`couchDB_service.js`

Responsible for querying CouchDB for issuer applications matching:

issuerType = subordinate
rootAuthorityId = currentRootAuthority
rootApprovalStatus = pending

Data Flow

Root Authority Dashboard
↓
apiService.getPendingSubordinatesForRoot()
↓
GET /api/root/subordinates/pending
↓
authController.getPendingSubordinatesForRoot()
↓
couchDB_service.getPendingSubordinatesForRoot()
↓
CouchDB issuers database

---

4. OPERATION: ROOT AUTHORITY APPROVES SUBORDINATE ISSUER

Purpose

Allows a Root Authority to approve a subordinate issuer application before it proceeds to final approval by the ChainCert administrator.

The approval is recorded in CouchDB and logged on the blockchain.

Files / Functions Involved

Frontend Service Layer

Function

`rootApproveSubordinateIssuer`

File

`apiService.js`

SDK Server Layer

Endpoint

POST /api/root-approve-subordinate/:issuerId

File

`sdk-server-fixed.js`

Controller Layer

Function

`rootApproveSubordinateIssuer`

File

`authController.js`

Responsibilities:

- Update root approval fields
- Call blockchain logging
- Return approval result

Database Layer

Function

`rootApproveSubordinateIssuer`

File

`couchDB_service.js`

Updates:

rootApprovalStatus = approved

rootApprovedBy

rootApprovedAt

Blockchain Layer

Chaincode Function

`LogRootSubordinateApproval`

File

`index.js`

Records approval event in blockchain audit log.

Data Flow

Root Authority Dashboard

↓

apiService.rootApproveSubordinateIssuer()

↓

POST /api/root-approve-subordinate/:issuerId

↓

authController.rootApproveSubordinateIssuer()

↓

couchDB_service.rootApproveSubordinateIssuer()

↓

Blockchain LogRootSubordinateApproval()

---

5. OPERATION: ROOT AUTHORITY REJECTS SUBORDINATE ISSUER

Purpose

Allows a Root Authority to reject a subordinate issuer application.

The rejection is stored in CouchDB and logged on the blockchain.

Files / Functions Involved

Frontend Service Layer

Function

`rootRejectSubordinateIssuer`

File

`apiService.js`

SDK Server Layer

Endpoint

POST /api/root-reject-subordinate/:issuerId

File

`sdk-server-fixed.js`

Controller Layer

Function

`rootRejectSubordinateIssuer`

File

`authController.js`

Database Layer

Function

`rootRejectSubordinateIssuer`

File

`couchDB_service.js`

Updates:

rootApprovalStatus = rejected
rootRejectionReason

Blockchain Layer

Chaincode Function

`LogRootSubordinateRejection`

File

`index.js`

---

6. OPERATION: ADMIN APPROVES ISSUER APPLICATION

Purpose

Final approval step that activates an issuer in the ChainCert network.

For subordinate issuers, this approval is only allowed **after Root Authority approval**.

Files / Functions Involved

Frontend Service Layer

`approveIssuer`

File

`apiService.js`

SDK Server Layer

POST /api/approve-issuer/:issuerId

File

`sdk-server-fixed.js`

Controller Layer

`approveIssuer`

File

`authController.js`

Database Layer

`approveIssuer`

File

`couchDB_service.js`

Updates:

status = approved
fabricIdentity
couchDBUrl
walletId

Blockchain Layer

`LogIssuerApproval`

File

`index.js`

---

7. OPERATION: ADMIN REJECTS ISSUER APPLICATION

Purpose

Allows the ChainCert administrator to reject issuer applications.

Files / Functions Involved

Frontend Service Layer

`rejectIssuer`

File

`apiService.js`

SDK Server Layer

POST /api/reject-issuer/:issuerId

File

`sdk-server-fixed.js`

Controller Layer

`rejectIssuer`

File

`authController.js`

Database Layer

`rejectIssuer`

File

`couchDB_service.js`

Updates:

status = rejected
rejectionReason
rejectedAt

Overall Issuer Governance Flow

Issuer Registration
↓
Issuer saved in CouchDB
↓
If subordinate → Root Authority approval required
↓
Root Authority approves or rejects
↓
If approved → ChainCert Admin approval
↓
Issuer becomes active in the network